Hamburg – Hospitals Shortest Path

In Python

Study Course

B208 Advanced Algorithms

By

Chehab Hany Mohamed Elsayed Elsayed Elmenoufi

Student Number: GH1034223

Under the Guidance of

Prof. Mahmoudreza Babaei

GitHub URL: https://github.com/Chippo90/Advanced-Algorithms

Gisma University of Applied Sciences

Berlin, Germany

March 2026

Table of Contents

1. Introduction *3*

2. System Architecture *3*

2.1 Overview 3

2.2 Responsibilities 4

2.2.1 Main 4

2.2.2 Data Loader 5

2.2.3 Graph Utils 5

2.2.4 Dijkstra 6

2.2.5 A* 7

2.2.6 Kruskal 7

2.2.7 Evaluation 7

3. Implementation *8*

3.1 Objective 8

3.2 Algorithms 8

3.3 Scenarios 8

4. Result Analysis *9*

5. Challenges and Solutions *10*

5.1 Challenge #1: Road Direction 10

5.2 Challenge #2: Large Distance Value 10

5.3 Challenge #3: Total hospitals number was 1 10

6. Conclusion and Future Work *10*

6.1 Conclusion 10

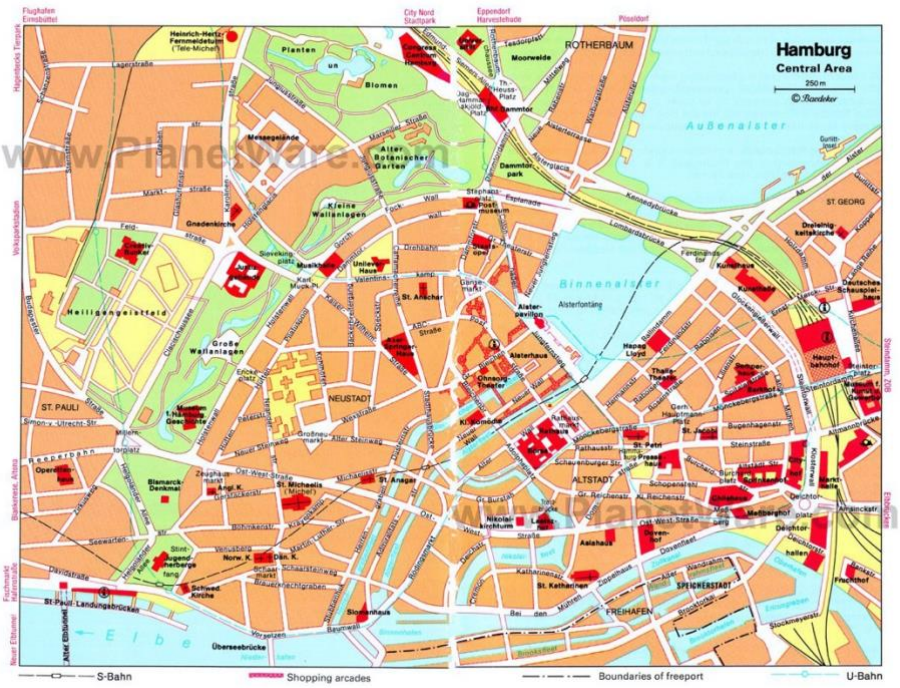
6.2 Future Work: 10

7. Research Limitations *10*

8. References *11*

1. Introduction

This project shows the road network of Hamburg as a weighted graph and applies greedy algorithms to compute the shortest path from random city locations to the nearest hospital, compare the performance of Dijkstra's algorithm and the A* algorithm and to analyze the network structure using Kruskal's Minimum Spanning Tree algorithm. Hamburg was chosen because it is a large metropolitan city with a complex road infrastructure, making it suitable for real-world algorithmic analysis.



2. System Architecture

2.1 Overview

The system is divided into multi files, each file is responsible for a single aspect of the system. This structure makes the project easier to understand and maintain:

- **main.py**: launches the application.
- **data_loader.py**: Download and process the OpenStreetMap data.("Google Maps vs. OpenStreetMap: Which One Is Better – WordPress Popup Plugin – FireBox," no date)
- **graph_utils**: Create list to store hospital nodes and extract name.
- **Dijkstra.py:** Compute shortest path using Dijkstra implementation.
- **astar.py**: Compute shortest path using A* implementation.
- **kruskal.py:** Compute Minimum Spanning Tree.
- **evaluation.py:** Apply scenarios from random location in the city.

2.2 Responsibilities

2.2.1 Main

- Import the modules from each file.

```python
#Import project modules
from data_loader import load_graph, load_hospitals
from graph_utils import map_hospitals_to_nodes
from dijkstra import dijkstra_nearest_hospital
from astar import astar_nearest_hospital
from kruskal import compute_kruskal_mst
from evaluation import run_experiments
```

- Load road network and hospitals.

```python
def main():
    #Load road network
    print("Loading graph...")
    G = load_graph()

    #Load hospitals
    print("Loading all hospitals...")
    hospitals = load_hospitals()
```

- Map the hospitals to graph nodes and extract their total number.

```python
#Map hospitals to graph nodes
print("Mapping hospitals to the graph...")
hospital_list = map_hospitals_to_nodes(G, hospitals)

#Print total hospitals
print(f"Total number of hospitals: {len(hospital_list)}")
```

- Run the scenarios.

```python
#Map hospitals to graph nodes
print("Mapping hospitals to the graph...")
hospital_list = map_hospitals_to_nodes(G, hospitals)

#Print total hospitals
print(f"Total number of hospitals: {len(hospital_list)}")
```

- Print all the 5 scenarios results.

```python
#Print results
print("\nResults:")

for r in results:
    print(f"\nScenario {r['scenario']}:")

    #Unpack Dijkstra results
    d_name, d_distance, d_time = r["dijkstra"]

    print("Dijkstra")
    print(f"  Hospital: {d_name}")
    print(f"  Distance: {d_distance:.2f} meters")
    print(f"  Runtime: {d_time:.4f} seconds")

    #Unpack A* results
    a_name, a_distance, a_time = r["astar"]

    print("A*")
    print(f"  Hospital: {a_name}")
    print(f"  Distance: {a_distance:.2f} meters")
    print(f"  Runtime: {a_time:.4f} seconds")
```

- Compute Kruskal MST.

```python
# Compute Kruskal MST
print("Computing Kruskal MST...")
mst = compute_kruskal_mst(G)

print("MST is computed successfully!")
```

2.2.2 Data Loader

All geographic data was collected from OpenStreetMap using OSMnx library. 2 datasets were retrieved: (*Find Shortest Path Using Python OSMnx Routing Module*, 12:22:19+00:00)

- The drive road network of Hamburg.

```python
#Import OSMnx to download and process OpenStreetMap data
import osmnx as ox

def load_graph(place_name="Hamburg, Germany"):  2 usages
    #Download the city drive roads network graph
    G = ox.graph_from_place(place_name, network_type="drive")

    #Project the graph in meters
    G = ox.project_graph(G)

    #Return the projected graph
    return G
```

- All nodes tagged to hospitals.

```python
def load_hospitals(place_name="Hamburg, Germany"):  2 usages
    #Define the OpenStreetMap tag used to retrieve hospitals
    tags = {"amenity": "hospital"}

    #Download all hospitals in the city
    hospitals = ox.features_from_place(place_name, tags)

    #Return the projected hospitals
    return hospitals
```

- OSMnx builds a graph representation of the road network automatically. ("OSMnx," no date)(*OpenStreetMap with OSMnx — NetworkX 3.6.1 documentation*, no date)

2.2.3 Graph Utils

The city was modeled as a graph:

- Nodes → Intersections.
- Edges → Roads.
- Each edge has weight = Length in meters.

```python
#Import OSMnx for nearest node search
import osmnx as ox

def map_hospitals_to_nodes(G, hospitals):  2 usages
    #Create an empty list to store hospital nodes
    #Each hospital is stored individually
    hospital_list = []

    #Loop for each hospital in the dataset
    for idx, row in hospitals.iterrows():

        #Compute the centroid of the hospital
        point = row.geometry.centroid

        #Extract hospital name
        name = row.get("name", "Unknown Hospital")

        #Find nearest graph node to hospital location
        nearest_node = ox.distance.nearest_nodes(G, point.x, point.y)

        #Store each hospital separately
        hospital_list.append((nearest_node, name))

    #Return hospital list
    return hospital_list
```

- Each hospital location was mapped to its nearest road network node using nearest search.
- Total hospitals in Hamburg is 31.(*German hospital directory*, no date)(*Hospitals in Hamburg*, no date)

2.2.4 Dijkstra

I chose it because it gurantees shortest path for graphs and it is used widely in navigation systems. (Kurniawan, Widyanto and Sukmasetya, 2024)

```python
import networkx as nx

def dijkstra_nearest_hospital(G, source, hospital_list):  2 usages

    #Convert graph to undirected to avoid direction issues
    G_undirected = G.to_undirected()

    #Compute shortest paths to all nodes
    lengths = nx.single_source_dijkstra_path_length(
        G_undirected,
        source,
        weight="length"
    )

    min_distance = float("inf")
    nearest_node = None
    nearest_name = None

    #Find nearest hospital among all nodes
    for node, name in hospital_list:
        if node in lengths:
            distance = lengths[node]
            if distance < min_distance:
                min_distance = distance
                nearest_node = node
                nearest_name = name

    return nearest_node, nearest_name, min_distance
```

The implementation of networkx library in my project is for creating and analyzing the complex network along with modeling the real data in a graph.(*NetworkX : Python software package for study of complex networks*, 10:25:37+00:00)

2.2.5 A*

I chose it because it's faster for spatial graphs and it's commonly used in navigation systems.(Cox, 2020)

```python
import networkx as nx

def heuristic(u, v, G):  1 usage
    #Get coordinates of node u
    x1, y1 = G.nodes[u]["x"], G.nodes[u]["y"]
    #Get coordinates of node v
    x2, y2 = G.nodes[v]["x"], G.nodes[v]["y"]
    #Return straight line distance
    return ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5

def astar_nearest_hospital(G, source, hospital_list):  2 usages
    #Convert graph to undirected
    G_undirected = G.to_undirected()

    min_distance = float("inf")
    nearest_node = None
    nearest_name = None

    for node, name in hospital_list:
        try:
            #Use A* to compute shortest path length to each hospital
            distance = nx.astar_path_length(
                G_undirected,
                source,
                node,
                heuristic=lambda u, v: heuristic(u, v, G_undirected),
                weight="length"
            )

            #If this is the smallest distance, update results
            if distance < min_distance:
                min_distance = distance
                nearest_node = node
                nearest_name = name

        except Exception as e:
            #If there's an issue, print error
            print(f"Error in A*: Source={source}, Hospital={name}: {e}")
            continue

    return nearest_node, nearest_name, min_distance
```

2.2.6 Kruskal

I chose it because it provides structural analysis and it demonstrates the behavior of greedy algorithms with shortest path.(*Time and Space Complexity Analysis of Kruskal Algorithm*, 11:10:37+00:00)

```python
#Import NetworkX
import networkx as nx

def compute_kruskal_mst(G):  2 usages
    #Convert graph to undirected
    G_undirected = G.to_undirected()

    #Compute Minimum Spanning Tree using
    mst = nx.minimum_spanning_tree(
        G_undirected,
        weight="length",
        algorithm="kruskal"
    )

    #Return MST graph
    return mst
```

2.2.7 Evaluation

I tried 5 scenarios, for each scenario:

- A random node in Hamburg was selected.
- Dijkstra calculated nearest hospital.
- A* calculated nearest hospital.

- Runtime was measured in seconds for both algorithms.
- Distance was measured in meters for both algorithms.

```python
import random
import time
import osmnx as ox

def run_experiments(G, hospital_list, dijkstra_func, astar_func):  2 usages

    results = []

    nodes = list(G.nodes)

    for i in range(5):

        #Choose random location in the city
        source = random.choice(nodes)

        #Dijkstra
        start = time.time()
        d_node, d_name, d_distance = dijkstra_func(
            G,
            source,
            hospital_list
        )
        d_time = time.time() - start

        #A*
        start = time.time()
        a_node, a_name, a_distance = astar_func(
            G,
            source,
            hospital_list
        )
        a_time = time.time() - start

        results.append({
            "scenario": i + 1,
            "source": source,
            "dijkstra": (d_name, d_distance, d_time),
            "astar": (a_name, a_distance, a_time)
        })

    return results
```

3. Implementation

3.1 Objective

The objective of this project is to solve the below problems:

- With a random location in Hamburg, we need to find the nearest hospitals on drive roads.
- Compare 2 shortest path algorithms with Dijkstra and A*.
- Make the analysis of the overall connectivity of the city road with Kruskal.

3.2 Algorithms

All implemented algorithms are greedy:

- Dijkstra selects the closest node.
- A* selects node f(n)=x(n) + y(n), where x(n) is the distance from source and y(n) is the distance to target.
- Kruskal selects smallest edge.

3.3 Scenarios

All applied scenarios were meant to:

- Extract the nearest hospital from the random node.
- Mention the distance in meters.
- Mention the runtime is seconds.

4. Result Analysis

- The implementation results start with loading the graph from OpenStreetMap

```
Loading graph...
```

- Next, will load the hospitals and map them to the loaded graph.

```
Loading graph...
Loading all hospitals...
Mapping hospitals to the graph...
```

- Last step, to count the number of total hospitals and to run the 5 scenarios.

```
Loading graph...
Loading all hospitals...
Mapping hospitals to the graph...
Total number of hospitals: 31
Running scenarios...
```

- Across all scenarios, Dijksta and A* found the same nearest hospital. This confirms that both algorithms are correct.

```
Scenario 1:
Dijkstra
  Hospital: Park-Klinik Blankenese
  Distance: 29479.12 meters
  Runtime: 3.0649 seconds
A*
  Hospital: Park-Klinik Blankenese
  Distance: 29479.12 meters
  Runtime: 4.9044 seconds
```

```
Scenario 2:
Dijkstra
  Hospital: Park-Klinik Blankenese
  Distance: 27570.81 meters
  Runtime: 2.5170 seconds
A*
  Hospital: Park-Klinik Blankenese
  Distance: 27570.81 meters
  Runtime: 4.4816 seconds
```

```
Scenario 3:
Dijkstra
  Hospital: Park-Klinik Blankenese
  Distance: 24508.94 meters
  Runtime: 2.7178 seconds
A*
  Hospital: Park-Klinik Blankenese
  Distance: 24508.94 meters
  Runtime: 4.8424 seconds
```

```
Scenario 4:
Dijkstra
  Hospital: Park-Klinik Blankenese
  Distance: 34657.73 meters
  Runtime: 2.5717 seconds
A*
  Hospital: Park-Klinik Blankenese
  Distance: 34657.73 meters
  Runtime: 4.6061 seconds
```

```
Scenario 5:
Dijkstra
  Hospital: Park-Klinik Blankenese
  Distance: 21391.68 meters
  Runtime: 2.5696 seconds
A*
  Hospital: Park-Klinik Blankenese
  Distance: 21391.68 meters
  Runtime: 3.7717 seconds
```

- Although, A* supposed to have a lower runtime but due to the map logic it was resulting a higher runtime than Dijkstra. A* is running 31 separate searches in every scenario while Dijkstra is running once.
- Algorithms Time and Space Complexity are as below:

Algorithm	Time Complexity	Space Compelxity
Dijksta	O(V+E) log V)	O(V)
A*	O(V+E) log V)	$O(V^E)$
Kruskal	O(E log E)	O(V+E)

- E→Number of edges in the graph.
- V→Number of vertices

5. Challenges and Solutions

5.1 Challenge #1: Road Direction

- **Description:** Some roads in OpenStreetMap are 1 way streets.
- **Solution:** The graph was converted to undirected graph during short path calculation.

5.2 Challenge #2: Large Distance Value

- **Description**: Shortest path distance were resulting in millions of meters.
- **Solution**: The graph was projected using G = ox.project_graph(G), this transformed the cordinates into metric system

5.3 Challenge #3: Total hospitals number was 1

- **Description**: Earlier, the hospital mapping logic was detecting only 1 hospital.
- **Solution**: I modified the mapping to hospital_list = [(node_id, hospital_name)], this ensured that each hospital remains in an individual graph node.

6. Conclusion and Future Work

6.1 Conclusion

The project successfully:

- Proved that graph algorithms remain highly applicable to real routing problems.
- Showed Hamburg as a weighted graph.
- Implemented and compared Dijkstra and A*.
- Demonstrated performance improvement using heuristics.
- Applied Kruskal to analyze network structure.

6.2 Future Work:

- Integrate visual map plotting to highlight paths.
- Use travel time instead of distance.
- Implement a third greedy algorithm and compare with others.
- Include traffic conditions and rush hours.

7. Research Limitations

- The network was computed in a static condition and no traffic data was implemented or put in consideration.
- The random source selection worked theoritically and didn't show the real density of population.
- The same hospital was chosen in every scenario due to it location in the heart of the city where many roads are leading to it.

---

8. References

Cox, G. (2020) *A* Pathfinding Algorithm | Baeldung on Computer Science*. Available at: https://www.baeldung.com/cs/a-star-algorithm (Accessed: March 22, 2026).

Find Shortest Path Using Python OSMnx Routing Module (12:22:19+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/python/find-shortest-path-using-python-osmnx-routing-module/ (Accessed: March 22, 2026).

German hospital directory (no date). Available at: https://www.german-hospital-directory.com/app/search/state/hamburg (Accessed: March 22, 2026).

"Google Maps vs. OpenStreetMap: Which One Is Better – WordPress Popup Plugin – FireBox" (no date) *FirePlugins.com*. Available at: https://www.fireplugins.com/blog/google-maps-vs-openstreetmap/ (Accessed: March 22, 2026).

Hospitals in Hamburg (no date). Available at: https://www.hamburg.com/residents/settle/hospitals-999320 (Accessed: March 20, 2026).

Kurniawan, F., Widyanto, R.A. and Sukmasetya, P. (2024) "Dijkstra Algorithm Implementation to Determine the Shortest Route to Hospital: A Case Study in Magelang District Indonesia," *E3S Web of Conferences*. Edited by M. Setiyo et al., 500, p. 01004. Available at: https://doi.org/10.1051/e3sconf/202450001004.

NetworkX : Python software package for study of complex networks (10:25:37+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/python/networkx-python-software-package-study-complex-networks/ (Accessed: March 22, 2026).

OpenStreetMap with OSMnx — NetworkX 3.6.1 documentation (no date). Available at: https://networkx.org/documentation/stable/auto_examples/geospatial/plot_osmnx.html (Accessed: March 22, 2026).

"OSMnx" (no date). Available at: https://osmnx.readthedocs.io/_/downloads/en/stable/pdf/ (Accessed: March 22, 2026).

Time and Space Complexity Analysis of Kruskal Algorithm (11:10:37+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-kruskal-algorithm/ (Accessed: March 22, 2026).